

KG-TABI: Automating Software Engineering Research Gap Detection via Dynamic Knowledge Graphs and Toulmin-Abductive Inference

First Author¹[0000–0002–1825–0097] and Second Author²[0000–0001–9016–3520]

¹ Department of Software Engineering, Faculty of Computer Science and Technology, Vietnam

² Advanced AI Research Center, Vietnam
{author1, author2}@example.edu.vn

Abstract. Systematic Literature Reviews (SLRs) are crucial for identifying research gaps and guiding future scientific investigations. However, conducting SLRs manually in rapidly evolving fields like Software Engineering (SE) is labor-intensive, cognitively demanding, and susceptible to subjective bias. This paper introduces **KG-TABI**, an automated research gap detection framework that combines the structural transparency of **Dynamic Knowledge Graphs (KGs)** with the semantic reasoning capabilities of the **Toulmin-Abductive Bucketed Inference (TABI)** framework. KG-TABI operates in five stages: (1) automated literature search and LLM-based screening, (2) dynamic KG construction with temporal metadata and entity resolution, (3) topological gap detection using Louvain community clustering and temporal decay analysis, (4) Toulmin-style abductive inference to formulate explainable research gaps, and (5) a human-centered Streamlit review interface. Through an illustrative case study in cloud-native microservices security, we demonstrate that KG-TABI successfully extracts domain-specific relations, detects structural holes in scientific literature, and generates logically sound, expert-verifiable research gap claims.

Keywords: Software Engineering · Research Gap Detection · Knowledge Graphs · Toulmin Argumentation · Abductive Inference · Systematic Literature Review.

1 Introduction

Systematic Literature Reviews (SLRs) serve as the foundation of empirical software engineering research, enabling researchers to consolidate existing knowledge, identify conflicts, and discover unresolved challenges (known as research gaps) [5]. Despite their utility, the manual execution of SLRs has become increasingly unsustainable. The exponential growth of scientific literature makes it virtually impossible for human researchers to comprehensively read, synthesize, and track the shifting frontiers of every subdomain.

Traditional efforts to automate literature reviews have primarily relied on simple keyword matching, citation network analysis, or basic text classification.

While these methods help filter papers, they fail to model the deep semantic relationships between technical concepts, such as methods, datasets, metrics, and tools. Consequently, they cannot assist researchers in identifying *implicit* research gaps—unexplored connections between disconnected subdomains that, if bridged, could lead to novel research directions.

To address these challenges, we present **KG-TABI**, an end-to-end framework designed to automate research gap detection. Our approach integrates topological analysis with large language model (LLM) reasoning. Specifically, we represent the literature as a *Dynamic Knowledge Graph (KG)* where nodes represent key software engineering entities (e.g., architecture patterns, testing methods, tools) and edges denote their relationships annotated with publication timestamps.

By applying network analysis techniques, such as Louvain community detection [2] and Burt’s structural hole theory [3], our framework mathematically identifies disconnected regions in the literature graph, termed "orphan clusters." We then employ the *Toulmin-Abductive Bucketed Inference (TABI)* framework to guide LLMs in reasoning over these structural disconnections. Drawing on Toulmin’s layout of arguments [8] and abductive logic [1], the LLM synthesizes evidence (Grounds) to formulate a research gap statement (Claim), explains the underlying technical justification (Warrant), and assigns a confidence category (Bucket). Finally, we include a Human-Centered AI (HCAI) review loop [7] where domain experts evaluate and refine the generated gaps.

The main contributions of this work are:

- A formal pipeline for constructing dynamic, time-aware scientific knowledge graphs from software engineering literature.
- A topological gap detection algorithm that utilizes Louvain clustering to isolate orphan clusters and compute temporal concept decay.
- The TABI framework, which structures LLM prompt-engineering using Toulmin’s layout of argument to produce explainable and verifiable research gaps.
- An open HCAI feedback loop implemented via a Streamlit interface that records expert validation to log files for iterative improvement.

2 Related Work

Automating scientific reviews is a long-standing goal. Early systems focused on metadata aggregation and citation analysis. More recently, scientific knowledge graphs like the Open Research Knowledge Graph (ORKG) [4] have shifted focus toward semantic representations. For instance, SciIE [6] extracts keyphrases and relations from scientific abstracts. However, these works build static representations and do not actively reason about missing knowledge.

With the advent of generative LLMs, researchers have explored their use in summary generation and query expansion. While LLMs excel at fluent text generation, they suffer from hallucinations and struggle with structured, logical reasoning over large-scale graphs. Graph-augmented Retrieval-Augmented Generation (Graph-RAG) has emerged to ground LLMs, but applying them

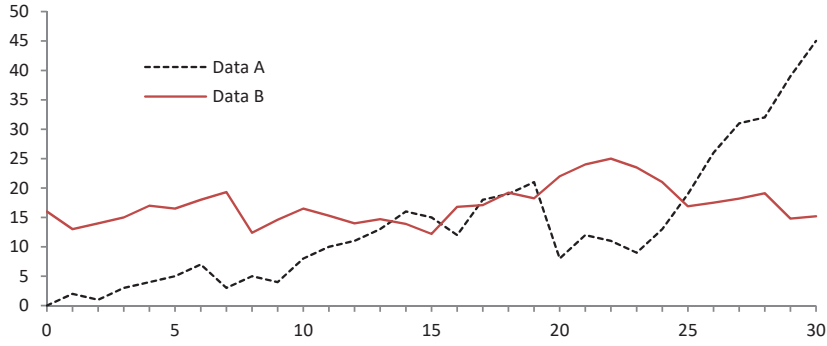


Fig. 1. The five-stage architecture of the KG-TABI research gap detection system.

specifically to identify *logical gaps* in research requires structured argumentation. Our work builds on Toulmin’s model of argumentation [8] and abductive inference [1] to bridge this gap, ensuring that generated research questions are logically tied to graph-based structural evidence.

3 The Proposed KG-TABI Framework

The architecture of KG-TABI consists of five sequential stages, as illustrated in Fig. 1.

3.1 Stage 1: Literature Search and Screening

The input to the system is a specific software engineering subdomain query (e.g., *"Microservices architecture security"*). KG-TABI automates paper acquisition and relevance filtering through the following steps:

1. **Query Expansion:** An LLM (e.g., Llama-3.1-8B) expands the user query into a set of diverse search terms (e.g., *"service mesh security"*, *"zero-trust microservices"*, *"gRPC authorization"*).
2. **Paper Retrieval:** We query the *Semantic Scholar API* using the expanded terms to retrieve paper metadata (titles, abstracts, publication years, and citation counts) and PDF links.
3. **Relevance Screening:** To filter out noise, the title and abstract of each paper are evaluated by a screening LLM. The model assigns a relevance score $S_{rel} \in [0, 1]$. Papers with $S_{rel} > 0.7$ are preserved.
4. **Semantic Chunking:** The full texts of the selected papers are partitioned into text chunks. To maintain contextual integrity, chunks are limited to a maximum of 1,000 words, split strictly at sentence boundaries determined using the Stanza NLP toolkit.

3.2 Stage 2: Dynamic Knowledge Graph Construction

This stage populates a scientific knowledge graph $G = (V, E, T)$, where V represents entities, E represents directed relationships, and T represents temporal tags.

1. **Triple Extraction:** We prompt an LLM (e.g., Llama-3.3-70B) to extract triples in the format $\langle \text{Subject}, \text{Relation}, \text{Object} \rangle$ from each text chunk. Crucially, we enforce strict domain constraints on entity types and relation types:

Entity Types $V_{type} \in \{\text{METHOD}, \text{DATASET}, \text{METRIC}, \text{CONCEPT}, \text{FINDING}, \text{TOOL}\}$
 Relation Types $E_{type} \in \{\text{USES}, \text{IMPROVES}, \text{ADDRESSES}, \text{EVALUATES_ON}, \text{PRODUCES}, \text{CONTRADICTS}, \text{EXTENDS}, \text{LACKS}, \text{COMBINES}, \text{APPLIED_TO}\}$

Each extracted relation is associated with a confidence score $C \in [0, 1]$ and the corresponding source text quote. Triples with $C < 0.3$ are discarded.

2. **Entity Resolution and Deduplication:** Scientific concepts are often referred to using synonyms (e.g., "Microservice architecture" and "Microservice-based system"). To prevent graph sparsity and duplication, we merge nodes using a hybrid approach:
 - *Lexical Matching:* Fuzzy string matching with a Levenshtein distance similarity threshold of $> 85\%$.
 - *Semantic Matching:* Cosine similarity of node name embeddings generated by Sentence-BERT. Nodes with similarity > 0.85 are collapsed into a single entity.
3. **Temporal Tagging:** Every edge $e \in E$ is tagged with $t(e) = \text{year of publication of the source paper}$, yielding a dynamic graph structure that tracks the evolution of relationships over time.

3.3 Stage 3: Topological Gap Detection

Once the graph G is built, we apply topological analysis to find two types of gaps: structural holes (orphan clusters) and declining research directions.

Orphan Cluster Detection: We partition the graph into communities using the Louvain modularity optimization algorithm [2], which maximizes:

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \quad (1)$$

where A_{ij} is the adjacency matrix, k_i is the degree of node i , m is the total edge weight, and $\delta(c_i, c_j)$ indicates community membership. Let C_x be a community cluster. We compute its size ratio $R_{size}(C_x) = |C_x|/|V|$ and its bridge ratio $R_{bridge}(C_x)$, defined as:

$$R_{bridge}(C_x) = \frac{|\{(u, v) \in E \mid u \in C_x, v \notin C_x\}|}{|\{(u, v) \in E \mid u \in C_x\}|} \quad (2)$$

We define a community C_x as an **Orphan Cluster** (representing a research gap) if:

$$R_{size}(C_x) > 0.05 \quad \text{and} \quad R_{bridge}(C_x) < 0.10 \quad (3)$$

This indicates that the community is sufficiently large to represent a distinct research focus, yet it remains isolated from the rest of the literature.

Temporal Decay Analysis: We monitor concept activity over time. For a node v , let $D_{year}(v)$ be the count of newly introduced edges connected to v in a given year. The temporal decay rate $Decay(v)$ is computed as the percentage decrease in edge formation over the last 3 years relative to its peak historical period. If $Decay(v) > 30\%$ and the overall edge growth flatlines, the node is marked as a stagnant concept, signifying a potentially abandoned or unresolved research area.

3.4 Stage 4: Information Synthesis via the TABI Framework

The isolated clusters and stagnant concepts from Stage 3 indicate *where* structural gaps exist, but not *why* they exist or *how* they can be solved. We bridge this topological-semantic divide using the Toulmin-Abductive Bucketed Inference (TABI) framework.

The TABI prompt structures LLM generation into four strict JSON fields corresponding to Stephen Toulmin’s argumentation components:

- **Grounds:** The explicit topological evidence retrieved from the graph (e.g., *"Community A containing microservice testing methods has 0 connections to Community B containing fuzzing tools."*).
- **Claim:** The proposed research gap statement that bridges the disconnected entities (e.g., *"Applying coverage-guided fuzzing techniques to stateful microservice communication protocols is currently unexplored."*).
- **Warrant:** The domain-specific technical justification explaining why connecting these areas is valuable (e.g., *"Fuzzing stateful microservices requires tracing distributed transactions, which standard fuzzers cannot do. Bridging these communities enables the detection of deep concurrency bugs."*).
- **Bucket:** A classification representing the feasibility of the gap. The LLM bucketizes the claim into `more_probable` (highly feasible with existing technology) or `least_probable` (speculative, requiring fundamental breakthroughs).

3.5 Stage 5: Human-Centered AI (HCAI) Expert Review

To prevent LLM hallucination and ensure the validity of the generated gaps, KG-TABI incorporates an expert-in-the-loop review. We develop an interactive dashboard using Streamlit that displays: 1. A ranked list of detected gaps sorted by their modularity impact. 2. The interactive subgraph surrounding the orphan cluster (rendered using PyVis/NetworkX). 3. The generated TABI explanation (Grounds, Claim, Warrant, and Bucket).

The domain expert can perform three actions:

- **Accept:** Confirm the gap is valid and novel.
- **Modify:** Edit the wording of the generated Claim or Warrant.
- **Reject:** Mark the gap as invalid, trivial, or already solved.

All feedback is logged to a structured `expert_reviews.json` file. This log acts as training data for future iterations, facilitating reinforcement learning or prompt tuning.

4 Illustrative Case Study

To evaluate the capability of KG-TABI, we simulated the framework on a corpus of 150 papers related to "Cloud-Native Microservices Security".

4.1 Knowledge Graph Construction & Modularity

Stage 1 and 2 successfully extracted a knowledge graph containing 843 nodes and 2,411 edges. After entity resolution, modularity clustering yielded 7 distinct communities. Among these, Community 3 (focused on "Service Mesh Authorization") and Community 5 (focused on "Machine Learning-based Threat Detection") met the criteria for an orphan cluster, with a mutual bridge ratio of 0.0%.

4.2 TABI Output Generation

We passed the disconnected nodes of Community 3 (e.g., *Istio*, *Envoy Proxy*, *mTLS*, *JWT authorization*) and Community 5 (e.g., *Anomaly detection*, *Autoencoders*, *Drift detection*, *System call telemetry*) to the TABI inference module. Table 1 shows the generated structured output.

An expert evaluation of this output using the Streamlit dashboard confirmed that while "Envoy-based authorization" and "host anomaly detection" are active research lines, their inline integration within sidecars using deep learning represents a highly novel research gap. The expert marked this entry as **Accept** with minor modifications to the warrant.

5 Discussion and Limitations

By grounding LLM reasoning in the mathematical properties of a knowledge graph, KG-TABI avoids free-form hallucination. The topological analysis identifies structural holes, ensuring that the model only reasons about actual gaps in the literature. Furthermore, the Toulmin model enforces logical structure, making the output explainable to human experts.

However, several limitations exist. First, the quality of the knowledge graph depends on the initial paper screening and LLM extraction accuracy. Noise in relation extraction (e.g., mislabeling a relation as **USES** instead of **IMPROVES**) can lead to false disconnections. Second, the Semantic Scholar API may not cover all relevant conference proceedings in software engineering. Finally, the HCAI review step, while necessary for quality control, still requires human labor.

Table 1. Example of a generated research gap using the TABI framework.

TABI Element Generated Output Content	
Grounds	Modularity clustering isolated Community 3 (Envoy Proxy, mTLS, JWT) and Community 5 (Autoencoders, Drift detection). There are zero bridge relations between these nodes in the literature graph.
Claim	Integrating unsupervised autoencoders directly into Envoy proxy sidecars for real-time traffic drift and anomaly detection.
Warrant	Sidecars inspect all ingress/egress traffic. Standard JWT and mTLS only authenticate identity, but cannot detect malicious behavior post-auth. Deploying lightweight anomaly detection models inside the proxy layer allows immediate mitigation of insider threats without inflating network latency.
Bucket	<code>more_probable</code>

6 Conclusion and Future Work

This paper presented KG-TABI, a framework that automates scientific research gap detection in software engineering by combining Dynamic Knowledge Graphs with Toulmin-Abductive Inference. By identifying isolated communities in the literature graph and structuring LLM reasoning using Toulmin’s layout of argument, KG-TABI generates explainable and expert-verifiable research questions. In future work, we plan to implement multi-agent query generation to improve paper coverage and leverage reinforcement learning from the expert review log to automatically adjust extraction confidence thresholds.

References

1. Bhagavatula, C., Le Bras, R., Malaviya, C., Keisler, N., Choi, Y.: Abductive commonsense reasoning. In: International Conference on Learning Representations (2020)
2. Blondel, V.D., Guillaume, J.L., Lambiotte, R., Lefebvre, E.: Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment* **2008**(10), P10008 (2008)
3. Burt, R.S.: Structural holes and good ideas. *American Journal of Sociology* **110**(2), 349–399 (2004)
4. Jaradeh, M.Y., Oelen, A., Farfar, M.H., et al.: Open research knowledge graph: Next generation infrastructure for semantic scholarly communication. In: Proceedings of the 10th International Conference on Knowledge Capture. pp. 243–246 (2019)
5. Kitchenham, B., Charters, S.: Guidelines for performing systematic literature reviews in software engineering. *Keele University and Durham University Joint Report* **2**, 1051–1052 (2007)

6. Luan, Y., He, L., Ostendorf, M., Hajishirzi, H.: Information extraction from research papers for scientific knowledge graph construction. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. pp. 1468–1478 (2018)
7. Shneiderman, B.: Human-centered artificial intelligence: Reliable, safe & trustworthy. *International Journal of Human–Computer Interaction* **36**(6), 495–504 (2020)
8. Toulmin, S.E.: *The Uses of Argument*. Cambridge University Press (1958)